

LEVEL 8
HIGHER DIPLOMA
IN SCIENCE IN
COMPUTING

ALGORITHMS AND
CONSTRUCTS

AZIZBEK ACHILOV
STUDENT ID: 2025381

CCT College Dublin

Assessment Cover Page

Module Title:	Algorithms and Constructs
Assessment Title:	CA2: Department Store Employee Management System
Lecturer Name:	Taufique Ahmed
Student Full Name:	Azizbek Achilov
Student Number:	2025381
Assessment Due Date:	23 May 2026
Date of Submission:	23 May 2026

Declaration

By submitting this assessment, I confirm that I have read the CCT policy on Academic Misconduct and understand the implications of submitting work that is not my own or does not appropriately reference material taken from a third party or other source. I declare it to be my own work and that all material from third parties has been appropriately referenced. I further confirm that this work has not previously been submitted for assessment by myself or someone else in CCT College Dublin or any other higher education institution.

TABLE OF CONTENTS:

1. INTRODUCTION	2
2. PROJECT OBJECTIVES	2
3. TECHNOLOGIES USED	3
4. SYSTEM FEATURES	3
5. REQUIREMENTS	6
6. DATA STRUCTURES AND ALGORITHMS.....	7
7. UML DIAGRAMS	7
8. USER STORIES	9
9. USE CASE DIAGRAM	10
10. UNIT TESTS	15
11. FLOWCHART	16
12. ALGORITHM JUSTIFICATION / CODE EXPLANATION	17
13. CONCLUSION	18
14. REFERENCES	19

1. INTRODUCTION

This project was developed to create a Department Store Management System using Java programming language. The purpose of the system is to manage employee information in an organized and efficient way.

The system allows users to add employees, search employee records, sort employee data, create a binary tree structure, and display employee hierarchy information. Different data structures and algorithms were implemented in the project to improve system performance and organization.

The project was designed using Object-Oriented Programming (OOP) concepts such as classes, objects, encapsulation, and modular programming. In addition, searching and sorting algorithms were implemented to improve data processing efficiency.

This report explains the system design, algorithms, implementation process, UML diagrams, flowcharts, and testing results of the project.

2. PROJECT OBJECTIVES

The main objectives of this project are:

- To develop a Department Store Management System using Java.
- To apply Object-Oriented Programming principles.
- To implement searching and sorting algorithms.
- To use binary tree data structures for employee hierarchy management.
- To perform employee data validation.
- To read employee information from external text files.
- To create a user-friendly console-based system.
- To improve understanding of data structures and algorithms through practical implementation.

3. TECHNOLOGIES USED

The following technologies and tools were used during the development of this project:

- Java Programming Language

The entire system was developed using Java because it supports Object-Oriented Programming and data structure implementation.

- Apache NetBeans IDE

NetBeans was used as the development environment for writing, testing, and running the program.

- Draw.io

Draw.io was used to create UML diagrams, use case diagrams, and flowcharts for system visualization.

- Text File Handling

The system reads employee data from an external text file using Java file handling classes.

4. SYSTEM FEATURES

The system contains several important features for employee management.

```
File read successfully!

===== DEPARTMENT STORE MANAGEMENT SYSTEM =====
1. SORT EMPLOYEES
2. SEARCH EMPLOYEE
3. ADD EMPLOYEE
4. CREATE BINARY TREE
5. DISPLAY TREE
6. EXIT
Enter your choice:
```

4.1 Add Employee

The user can add new employee information into the system. Employee details include employee name, manager type, and department.

```
Output - Run (CA_2) x
5. DISPLAY TREE
6. EXIT
Enter your choice: 3
Enter employee name: Aziz
Select Manager Type:
1. HEAD_MANAGER
2. ASSISTANT_MANAGER
3. TEAM_LEAD
1
Select Department:
1. SALES
2. HR
3. IT
4. CUSTOMER_SERVICE
2
Employee added successfully!
```

4.2 Search Employee

The system allows users to search for employees using binary search algorithms for faster searching performance.

```
Output - Run (CA_2) x
2. SEARCH EMPLOYEE
3. ADD EMPLOYEE
4. CREATE BINARY TREE
5. DISPLAY TREE
6. EXIT
Enter your choice: 2
Enter employee name to search: Aziz

===== EMPLOYEE FOUND =====

Employee Information
-----
Name: Aziz
Manager Type: HEAD_MANAGER
Department: HR
```

4.3 Sort Employees

Employee records can be sorted alphabetically using merge sort algorithms.

```
Output - Run (CA_2) x
===== DEPARTMENT STORE MANAGEMENT SYSTEM =====
1. SORT EMPLOYEES
2. SEARCH EMPLOYEE
3. ADD EMPLOYEE
4. CREATE BINARY TREE
5. DISPLAY TREE
6. EXIT
Enter your choice: 1

===== SORTED EMPLOYEES =====

Employee Information
-----
Name: Alex
Manager Type: TEAM_LEAD
Department: IT

Employee Information
-----
Name: Ali
Manager Type: TEAM_LEAD
Department: SALES

Employee Information
-----
Name: Amelia
Manager Type: TEAM_LEAD
Department: CUSTOMER_SERVICE

Employee Information
-----
Name: Aziz
```

4.4 Create Binary Tree

The system creates a binary tree structure to organize employee hierarchy information.

```
Output - Run (CA_2) x
===== DEPARTMENT STORE MANAGEMENT SYSTEM =====
1. SORT EMPLOYEES
2. SEARCH EMPLOYEE
3. ADD EMPLOYEE
4. CREATE BINARY TREE
5. DISPLAY TREE
6. EXIT
Enter your choice: 4
Binary Tree Created Successfully!
```

4.5 Display Employee Tree

The binary tree hierarchy can be displayed together with tree height and node count information.

```
Output - Run (CA_2) x
Manager Type: TEAM_LEAD
Department: IT

Employee Information
-----
Name: Sophia
Manager Type: TEAM_LEAD
Department: SALES

Employee Information
-----
Name: Vali
Manager Type: HEAD_MANAGER
Department: HR

Employee Information
-----
Name: Victor
Manager Type: TEAM_LEAD
Department: IT

Total Nodes: 50
Tree Height: 6
```

4.6 File Reading

Employee information can be loaded automatically from an external text file.

```
File read successfully!

===== DEPARTMENT STORE MANAGEMENT SYSTEM =====
1. SORT EMPLOYEES
2. SEARCH EMPLOYEE
3. ADD EMPLOYEE
4. CREATE BINARY TREE
5. DISPLAY TREE
- 6. EXIT
Enter your choice:
```

4.7 Validation

The system validates user input to prevent invalid menu options and incorrect data entry.

```
===== DEPARTMENT STORE MANAGEMENT SYSTEM =====  
1. SORT EMPLOYEES  
2. SEARCH EMPLOYEE  
3. ADD EMPLOYEE  
4. CREATE BINARY TREE  
5. DISPLAY TREE  
6. EXIT  
Enter your choice: 99  
Invalid menu option!
```

5. REQUIREMENTS

Functional Requirements

ID	Functional Requirement	Reason
FR-01	The system shall allow users to add employees	To maintain employee records
FR-02	The system shall sort employees alphabetically	To improve organization
FR-03	The system shall search employees using binary search	To improve search efficiency
FR-04	The system shall validate all user input	To prevent invalid data
FR-05	The system shall create a binary tree hierarchy	To visualize employee structure

Non-Functional Requirements

ID	Non-Functional Requirement	Reason
NFR-01	The system should respond within 2 seconds	Performance
NFR-02	The system should prevent invalid input crashes	Reliability
NFR-03	The system should support future scalability	Scalability
NFR-04	The console menu should be user-friendly	Usability
NFR-05	The code should follow modular OOP design	Maintainability

6. DATA STRUCTURES AND ALGORITHMS

Different data structures and algorithms were implemented in this project to improve system efficiency and organization.

5.1 Binary Tree

A binary tree structure was used to organize employee hierarchy information. Each node stores employee data and links to left and right child nodes.

5.2 Merge Sort

Merge sort algorithm was used to sort employee records alphabetically. Merge sort is efficient for handling large amounts of data because of its divide-and-conquer approach.

5.3 Binary Search

Binary search algorithm was used to quickly search employee records. Binary search improves searching performance compared to linear searching methods.

5.4 ArrayList

ArrayList was used to store employee records dynamically during program execution.

7. UML DIAGRAMS

The UML class diagram below shows the structure of the system classes and the relationships between them.

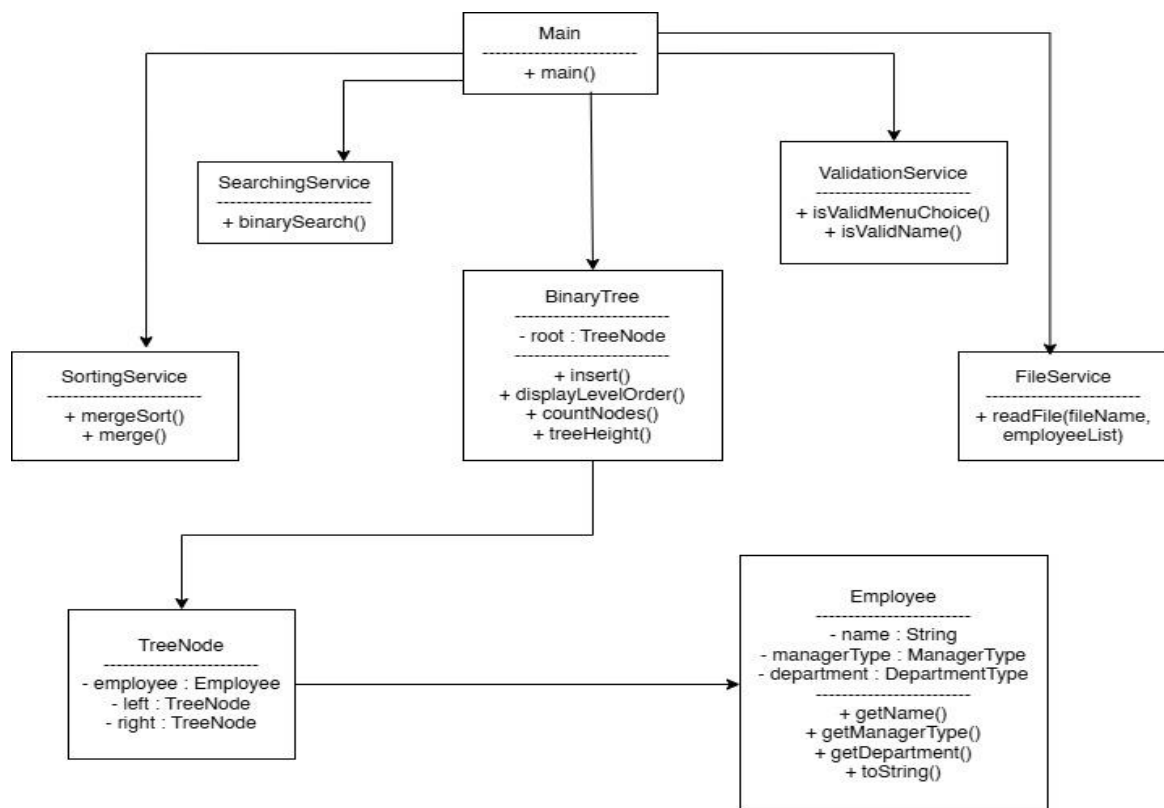


Figure 1: UML Class Diagram

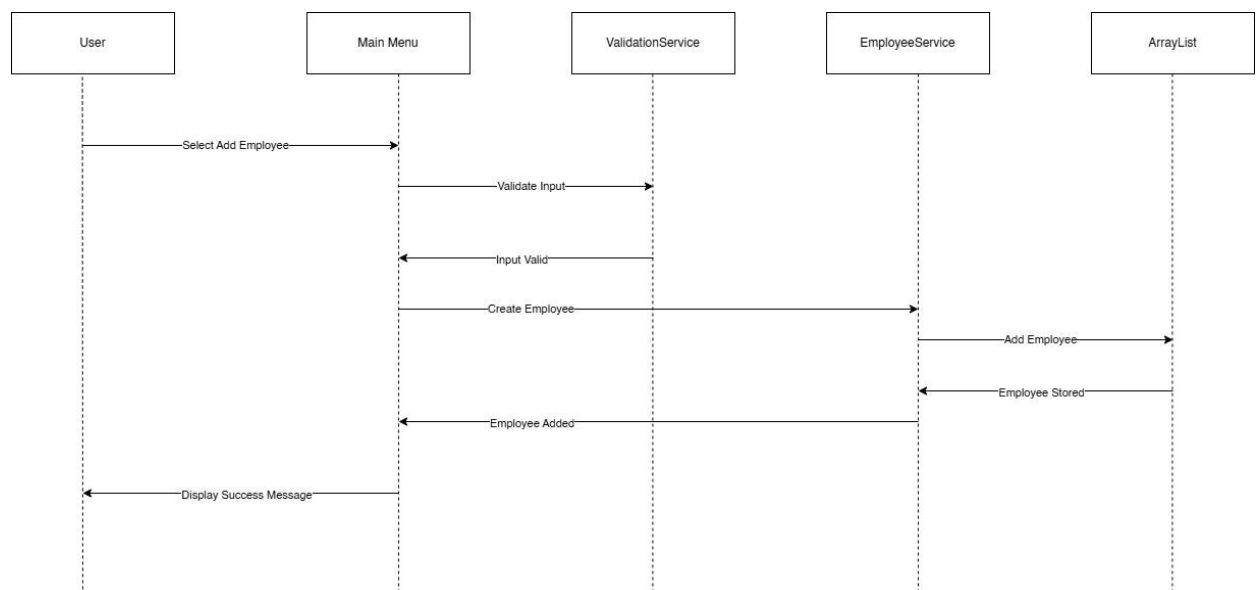


Figure 2: Add Employee Sequence Diagram

The sequence diagram above illustrates the interaction between different system components during the employee creation process. The user selects the “Add Employee” option from the main menu, after which the system validates the entered information using the ValidationService class.

Once the validation process is completed successfully, the EmployeeService creates the employee record and stores it inside the ArrayList structure. Finally, the system displays a success confirmation message to the user. This diagram demonstrates communication between objects using object-oriented design principles.

8. USER STORIES

USER STORY 1 — Sort Employees

As a store manager, I want to sort employees alphabetically so that employee information can be viewed in an organised way.

Acceptance Criteria:

- The user can select the sort option from the menu.
- The system performs merge sort correctly.
- Employees are displayed alphabetically.
- Sorted output is readable and clear.

USER STORY 2 — Search Employee

As a store manager, I want to search employee records quickly so that I can find employee information without checking the full list manually.

Acceptance Criteria:

- The user can enter an employee name.
- The system performs binary search.
- Employee details are displayed correctly.
- If the employee does not exist, a message is displayed.

USER STORY 3 — Add Employee

As a store manager, I want to add new employees into the system so that employee records remain updated.

Acceptance Criteria:

- The user can enter employee details.
- Invalid department or manager type is rejected.
- Valid employee data is saved successfully.
- Confirmation message is displayed after successful addition.

USER STORY 4 — Create Binary Tree

As a store manager, I want to create an employee hierarchy tree so that employee structure can be visualised clearly.

Acceptance Criteria:

- Employee records are inserted into the tree.
- Tree generation completes successfully.
- Employee hierarchy is displayed properly.
- Tree height and node count are shown.

USER STORY 5 — Read Employee File

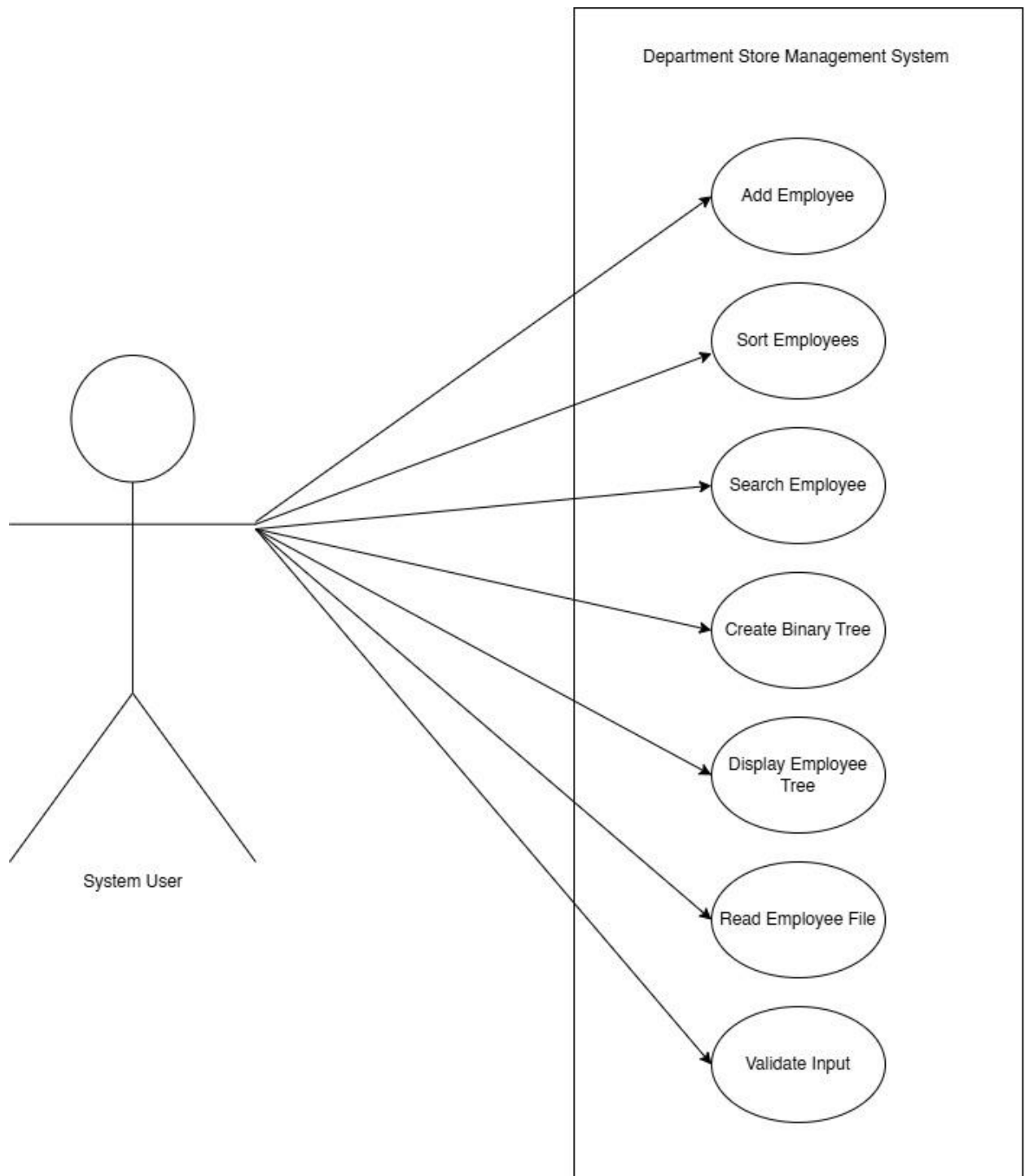
As a system user, I want employee data to load automatically from a text file so that manual data entry is reduced.

Acceptance Criteria:

- The system reads the text file successfully.
- Employee records are loaded into memory.
- Invalid lines are skipped safely.
- A success message is displayed after loading.

9. USE CASE DIAGRAM

The use case diagram below illustrates how the user interacts with the Department Store Management System.



Use Case Narrative / Use Case Specification

UC-01 - Sort Employees

Use Case Name: Sort Employees

Description: This use case allows the user to sort employee records alphabetically using the merge sort algorithm.

Dependencies: Employee records must already exist in the system.

Preconditions:

- The application is running.
- Employee data has been loaded successfully.

Primary Actor: System User

Main Flow:

1. The user selects the “Sort Employees” option from the main menu.
2. The system applies the merge sort algorithm.
3. Employee records are arranged alphabetically.
4. The sorted employee list is displayed on the screen.

Postconditions: Employees are displayed in correct alphabetical order.

Alternative Flow: If no employee records are available, the system displays an appropriate message.

Notes: Recursive merge sort is used because it provides efficient sorting performance.

UC-02 - Search Employee

Use Case Name: Search Employee

Description: This use case allows the user to search for an employee by name using binary search.

Dependencies: The employee list must already be sorted.

Preconditions:

- Employee records are loaded.
- The employee list is sorted alphabetically.

Primary Actor: System User

Main Flow:

1. The user selects “Search Employee”.
2. The user enters the employee name.
3. The system performs binary search.
4. Matching employee information is displayed.

Postconditions: Requested employee details are shown to the user.

Alternative Flow: If the employee cannot be found, the system displays a “Not Found” message.

Notes: Binary search was implemented to improve searching speed.

UC-03 - Add Employee

Use Case Name: Add Employee

Description: This use case allows the user to add a new employee into the system.

Dependencies: The employee list must be active.

Preconditions:

- The system is running.
- The user is in the main menu.

Primary Actor: System User

Main Flow:

1. The user selects “Add Employee”.
2. The user enters employee details.
3. The system validates department and manager type.
4. The employee is added to the employee list.
5. The system confirms successful addition.

Postconditions: A new employee record is stored successfully.

Alternative Flow: If invalid input is entered, the system displays an error message and asks the user to re-enter the information.

Notes: Input validation is used to improve data accuracy.

UC-04 - Create Binary Tree

Use Case Name: Create Binary Tree

Description: This use case generates an employee hierarchy using a binary tree structure.

Dependencies: Employee records must exist in the system.

Preconditions:

- Employee data has already been loaded.

Primary Actor: System User

Main Flow:

1. The user selects “Create Binary Tree”.
2. The system inserts employees into the binary tree.
3. The employee hierarchy is generated.
4. Tree information is displayed.

Postconditions: The binary tree structure is successfully created.

Alternative Flow: If the employee list is empty, the system displays a warning message.

Notes: Binary tree structure improves hierarchy organisation and traversal.

UC-05 - Display Employee Tree

Use Case Name: Display Employee Tree

Description: This use case displays the employee hierarchy tree together with tree statistics.

Dependencies: The binary tree must already exist.

Preconditions:

- The binary tree has been generated successfully.

Primary Actor: System User

Main Flow:

1. The user selects “Display Employee Tree”.
2. The system performs level-order traversal.
3. Employee hierarchy is displayed.
4. Tree height and node count are displayed.

Postconditions: Hierarchy information becomes visible to the user.

Alternative Flow: If the tree has not been created, the system displays an appropriate message.

Notes: Level-order traversal is used for clear hierarchy visualization.

UC-06 - Exit System

Use Case Name: Exit System

Description: This use case allows the user to safely close the application.

Dependencies: None

Preconditions:

- The application is active.

Primary Actor: System User

Main Flow:

1. The user selects “Exit”.
2. The application closes successfully.

Postconditions: The system is terminated safely.

Alternative Flow: None

Notes: Program resources are released during shutdown.

10.UNIT TESTS

Test Case	Purpose	Input	Expected Result	Actual Result	Status
Merge Sort Test	Verify employee sorting	Unsorted employee list	Employees sorted alphabetically	Employees sorted correctly	PASS
Binary Search Test	Verify employee searching	“Alex”	Employee details displayed	Employee found successfully	PASS
Validation Test	Verify invalid department handling	Invalid department	Error message displayed	Validation worked correctly	PASS
File Reading Test	Verify text file loading	Applicants_Form.txt	Employee records loaded	File loaded successfully	PASS
Binary Tree Test	Verify tree generation	Employee list	Tree hierarchy displayed	Tree generated correctly	PASS

Several tests were performed to ensure the system works correctly.

- Employee sorting was tested successfully.
- Employee searching functionality returned correct search results.
- Invalid input validation prevented incorrect menu selections.
- Binary tree creation displayed correct hierarchy information.
- File reading functionality loaded employee data successfully.

The testing results confirmed that the system functions as expected.

11. FLOWCHART

The flowchart below explains the overall system workflow and program execution process.

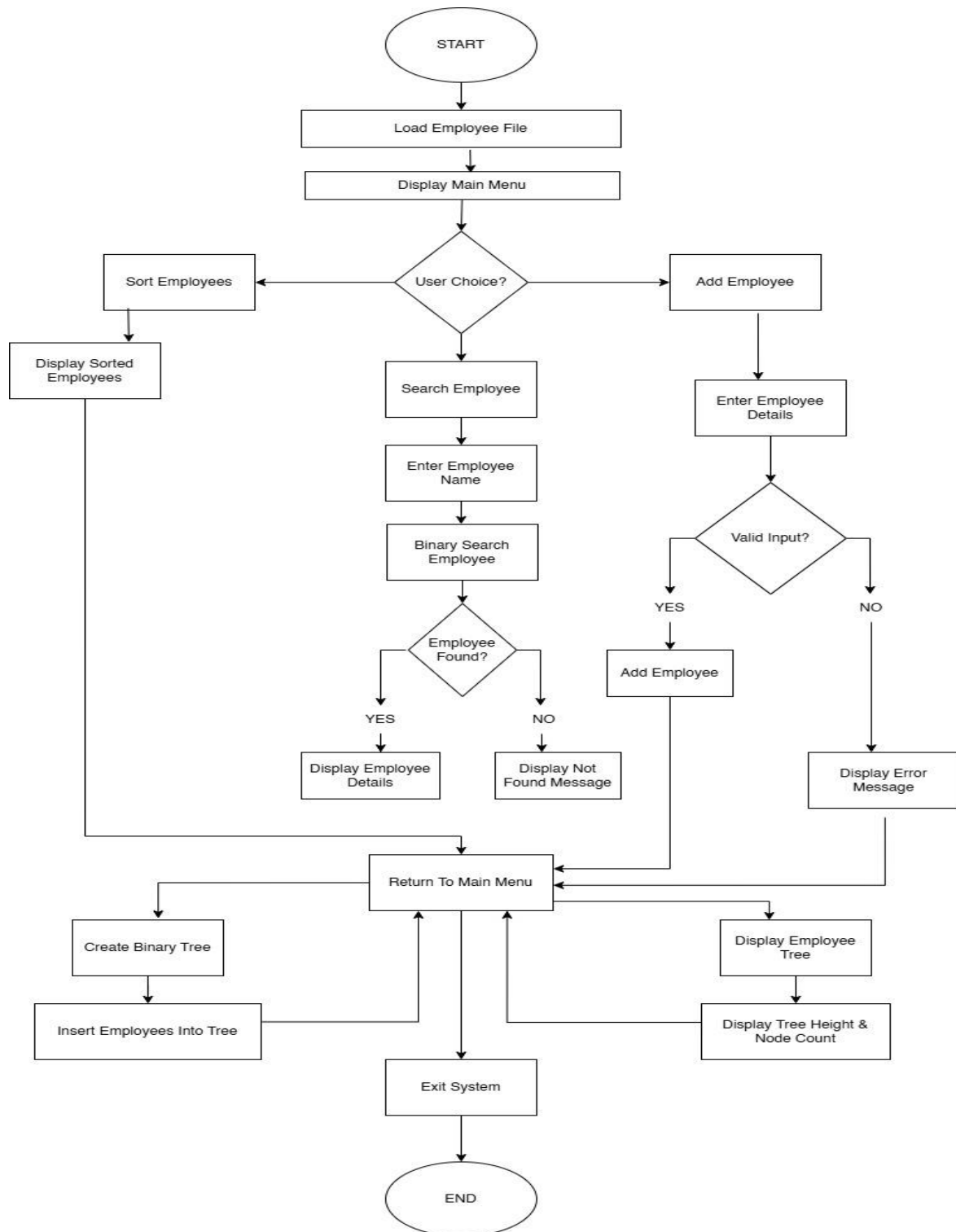


Figure 3: System Workflow Flowchart

12. ALGORITHM JUSTIFICATION / CODE EXPLANATION

MERGE SORT

Merge Sort was selected for employee sorting because it is an efficient recursive sorting algorithm with good performance for larger datasets. The algorithm divides the employee list into smaller parts and merges them back together in sorted order.

Merge Sort was preferred over simpler algorithms such as Bubble Sort because it provides better scalability and faster performance when the number of employee records increases.

The algorithm also maintains stable sorting behaviour, which helps preserve employee ordering consistency during sorting operations.

BINARY SEARCH

Binary Search was implemented to improve searching speed inside the employee management system. Instead of checking every employee one by one, the algorithm repeatedly divides the search range in half until the employee is found.

Binary Search works efficiently together with Merge Sort because the employee list remains alphabetically sorted.

This approach significantly reduces search time and improves system performance compared to linear searching methods.

BINARY TREE

Binary Tree data structure was used to organise employee hierarchy information inside the system. Each TreeNode stores employee data together with references to left and right child nodes.

The binary tree allows the system to display employee hierarchy using level-order traversal while also calculating tree height and node count.

This structure improves data organisation and demonstrates practical implementation of tree-based data structures in Java.

FILE HANDLING

File handling was implemented to automatically load employee records from an external text file. This feature reduces manual data entry and improves usability.

The system reads each line from the file, separates employee details, validates the information, and stores the data inside an ArrayList.

Invalid or incomplete records are skipped safely to prevent program crashes and improve system reliability.

INPUT VALIDATION

Input validation was implemented to prevent incorrect department and manager type entries. Validation helps maintain consistent employee data and reduces user mistakes.

The system checks menu choices, department values, and manager type selections before storing employee information.

This improves system reliability, usability, and overall data accuracy.

ALGORITHM COMPARISON

Algorithm	Complexity	Why Selected
Merge Sort	$O(n \log n)$	Better scalability
Bubble Sort	$O(n^2)$	Too slow for larger datasets
Binary Search	$O(\log n)$	Faster than linear search
Linear Search	$O(n)$	Less efficient

13.CONCLUSION

In conclusion, the Department Store Management System was successfully designed and implemented using Java programming language together with different data structures and algorithms.

The project demonstrates practical use of Object-Oriented Programming concepts such as classes, objects, encapsulation, modular design, and separation of responsibilities between system components.

Several important algorithms and data structures were implemented during development, including Merge Sort, Binary Search, ArrayList, and Binary Tree structures. These features improved employee management, searching performance, sorting efficiency, and hierarchy organisation.

The system also includes file handling, input validation, UML diagrams, use case specifications, user stories, flowcharts, and unit testing to support software design and software engineering practices.

Throughout the development process, the project improved understanding of recursive algorithms, data organisation techniques, system design principles, and Java application development.

Overall, the project achieved all planned objectives successfully and provided a functional employee management solution for a department store environment.

14. REFERENCES

Oracle. (2025) Java Documentation. Available at:

<https://docs.oracle.com/javase/>

GeeksforGeeks. (2025) Merge Sort Algorithm. Available at:

<https://www.geeksforgeeks.org/merge-sort/>

GeeksforGeeks. (2025) Binary Search Algorithm. Available at:

<https://www.geeksforgeeks.org/binary-search/>

GeeksforGeeks. (2025) Binary Tree Data Structure. Available at:

<https://www.geeksforgeeks.org/binary-tree-data-structure/>

Oracle. (2025) Java File Handling. Available at:

<https://docs.oracle.com/javase/tutorial/essential/io/file.html>

W3Schools. (2025) Java ArrayList. Available at:

https://www.w3schools.com/java/java_arraylist.asp

JavaTPoint. (2025) Java OOP Concepts. Available at:

<https://www.javatpoint.com/java-oops-concepts>

Draw.io. (2025) Diagramming Tool. Available at:

<https://app.diagrams.net/>

Github link:

https://github.com/2025381-Aziz/CA2_DepartmentStoreManagement